

Q.No :4

Question Description:

Ragu has given a range $[L, R]$ to Smith. Smith wants to require to find the number of integers 'X' in the range such that $\text{GCD}[X, F[X]] > 1$ where $F[x]$ is equal to the sum of digits of 'X' in its hexadecimal (or base 16) representation.

Example : $F[27] = 1+8=1+11=12$

(27 in hexadecimal is written as 1B)

Constraints:

$1 \leq T \leq 50$

$1 \leq L$

$R \leq 10^5$

Input Format:

The first line contains a positive integer 'T' denoting the number of questions that you are asked.

Each of the next 'T' lines contain two integers 'L' and 'R' denoting the range of questions.

Output Format:

Print the output in a separate lines exactly 'T' numbers as the output.

Sample Input:

```
3
1 3
5 8
7 12
```

Sample output:

```
2
4
6
```

Explanation

For the first test-case, numbers which satisfy the criteria specified are : 2,3. Hence, the answer is 2.

For the second test-case, numbers which satisfy the criteria specified are : 5,6,7 and 8. Hence, the answer is 4.

For the third test-case, numbers which satisfy the criteria specified are : 7,8,9,10,11 and 12. Hence, the answer is 6.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3
1 3
5 8
7 12
```

Test Case 2

INPUT (STDIN)

```
10
15 64
14 100
9 79
19 63
3 83
5 61
18 93
```

EXPECTED OUTPUT

```
2
4
6
```

EXPECTED OUTPUT

```
20 98
13 71
12 53
```

EXPECTED OUTPUT

```
29
52
45
26
53
37
45
47
36
26
```

Code:

```
#include <stdio.h>
int gcd(int a, int b)
{
    while (b != 0)
    {
        int temp = a % b;
```

```
a = b;
b = temp;
}

return a;
}

int hexSum(int n)
{
    int sum = 0;

    while (n > 0)
    {
        sum = sum + (n % 16);
        n = n / 16;
    }

    return sum;
}

int search(int a, int b)
{
    int count = 0;

    for (int x = a; x <= b; x++)
    {
        int sum = hexSum(x);

        if (gcd(x, sum) > 1)
        {
            count++;
        }
    }

    return count;
}

int main()
{
    int T;
    int L, R;

    scanf("%d", &T);

    for (int i = 0; i < T; i++)
    {
        scanf("%d %d", &L, &R);
```

```
printf("%d\n", search(L, R));  
}  
  
return 0;  
}
```

Output:

Test_case 1

```
3  
1 3  
2  
5 8  
4  
7 12  
6
```

Test_case 2:

```
10  
15 64  
29  
14 100  
52  
9 79  
45  
19 63  
26  
3 83  
53  
5 61  
37  
18 93  
45  
20 98  
47  
13 71  
36  
12 53  
26
```